MANUAL DE INSTALACIÓN

DE ENTORNO DE DESARROLLO

(Git, Node.js, Python, Codex, Archify, Flutter y Docker con WSL)



DATOS DEL DOCUMENTO	DETALLE
Autor	DIMATREK
Fecha de elaboración	19 de octubre de 2026
Versión	1.0
Plataforma objetivo	Windows 10/11 de 64 bits
Tipo de documento	Manual de instalación y configuración

Guía técnica basada en comandos y evidencias visuales de instalación

Control del documento

Elemento	Descripción
Objetivo	Documentar la instalación, verificación y configuración inicial de herramientas para desarrollo de software y contenedorización en Windows.
Audiencia	Estudiantes, desarrolladores y personal técnico que preparan una estación de trabajo Windows.
Alcance	Git, Node.js LTS, Python, OpenAI Codex CLI, skills de Archify, Flutter SDK, Docker Desktop y WSL con Ubuntu.
Convenciones	Ejecutar comandos en la terminal indicada. Las salidas de consola incluidas son ejemplos esperados y pueden variar según la versión o el equipo.
Seguridad	No compartir contraseñas, tokens, claves privadas ni credenciales. Ejecutar scripts de instalación únicamente después de verificar que proceden de la fuente oficial.

Contenido

Control del documento 2

1. Introducción y alcance 5

1.1 Requisitos previos 5

2. Preparación del sistema 5

2.1 Comprobación de Winget 5

3. Git 6

3.1 Descripción y requisitos 6

3.2 Instalación 6

3.3 Verificación 6

3.4 Configuración global de identidad 6

3.5 Comprobar la configuración 6

4. Node.js (LTS) 8

4.1 Descripción y requisitos 8

4.2 Instalación 8

4.3 Verificación 8

5. Python 9

5.1 Descripción y requisitos 9

5.2 Instalación 9

5.3 Verificación 9

6. OpenAI Codex CLI 10

6.1 Descripción y requisitos 10

6.2 Instalación mediante PowerShell 10

6.3 Verificación y prueba interactiva 10

7. Archify: instalación de skills y agentes 12

7.1 Descripción y requisitos 12

7.2 Instalación de skills 12

7.3 Selección de agentes y método 12

7.4 Verificación 12

8. Flutter SDK 15

8.1 Descripción y requisitos 15

8.2 Descarga y extracción 15

8.3 Verificación 15

9. Docker Desktop y WSL 17

9.1 Descripción y requisitos 17

9.2 Instalación de Docker Desktop 17

9.3 Activar características de Windows 18
9.4 Resolver el error “WSL is not installed” 18
9.5 Configuración inicial de Ubuntu 19
9.6 Verificación de WSL y Docker 21
9.7 Integración de Docker Desktop con WSL 21

10. Dockerizar una aplicación 23

10.1 Archivos Dockerfile y .dockerignore 23
10.2 Ejemplo de .dockerignore 23
10.3 Ejemplo de Dockerfile 24
10.4 Compilación del APK (móvil) 24

11. Verificación integral 31

12. Solución de problemas frecuentes 32

13. Consideraciones de seguridad y mantenimiento 32

Referencias oficiales 32

1. Introducción y alcance

Este manual describe el procedimiento para preparar un entorno de desarrollo en Windows mediante herramientas de control de versiones, ejecución de JavaScript, programación con Python, asistencia de programación por línea de comandos, instalación de habilidades para agentes, desarrollo multiplataforma con Flutter y contenedorización con Docker Desktop apoyado en WSL.

La guía se elaboró a partir de los comandos y capturas incluidos en el documento de referencia. Las versiones indicadas corresponden a las evidencias proporcionadas y no garantizan que sean las versiones más recientes disponibles en futuras fechas.

1.1 Requisitos previos

- Equipo con Windows 10 u 11 actualizado y arquitectura compatible (las capturas muestran Windows x64).
- Conexión a Internet para descargar instaladores y componentes.
- Acceso a una terminal: Símbolo del sistema (CMD) o PowerShell.
- Permisos de administrador cuando Windows solicite habilitar características del sistema o instalar componentes.
- Espacio libre suficiente para SDK, imágenes de contenedores y distribuciones Linux.

2. Preparación del sistema

Antes de iniciar, abra CMD o PowerShell. Para habilitar características de Windows y solucionar la instalación de WSL, utilice PowerShell como administrador. Tras instalar herramientas, abra una terminal nueva para que se actualice la variable PATH.

2.1 Comprobación de Winget

```
winget --version
```

Resultado esperado: un número de versión de Windows Package Manager. Si el comando no se reconoce, actualice o instale App Installer desde Microsoft Store y vuelva a abrir la terminal.

3. Git

3.1 Descripción y requisitos

Git es un sistema distribuido de control de versiones. Permite registrar cambios, crear ramas, comparar versiones y colaborar mediante repositorios remotos. La instalación se realiza con Winget.

3.2 Instalación

```
winget install --id Git.Git -e --source winget
```

En las capturas, Winget encuentra Git versión 2.55.0.3 y finaliza con el mensaje “Successfully installed”.

3.3 Verificación

```
git --version
git version 2.55.0.windows.3
```

La salida real puede incluir el sufijo de compilación de Windows. La captura muestra Git 2.55.0.windows.3, correspondiente al paquete 2.55.0.3.

3.4 Configuración global de identidad

```
git config --global user.name "Derek Sesni"
git config --global user.email "230892@utxicotepec.edu.mx"
```

Estos valores se aplican a los repositorios del usuario de Windows y se usan como autoría predeterminada de los commits. Sustituya el correo si desea usar otra identidad.

3.5 Comprobar la configuración

```
git config --global user.name
git config --global user.email
```

Resultado esperado: “Derek Sesni” y “230892@utxicotepec.edu.mx”, respectivamente.

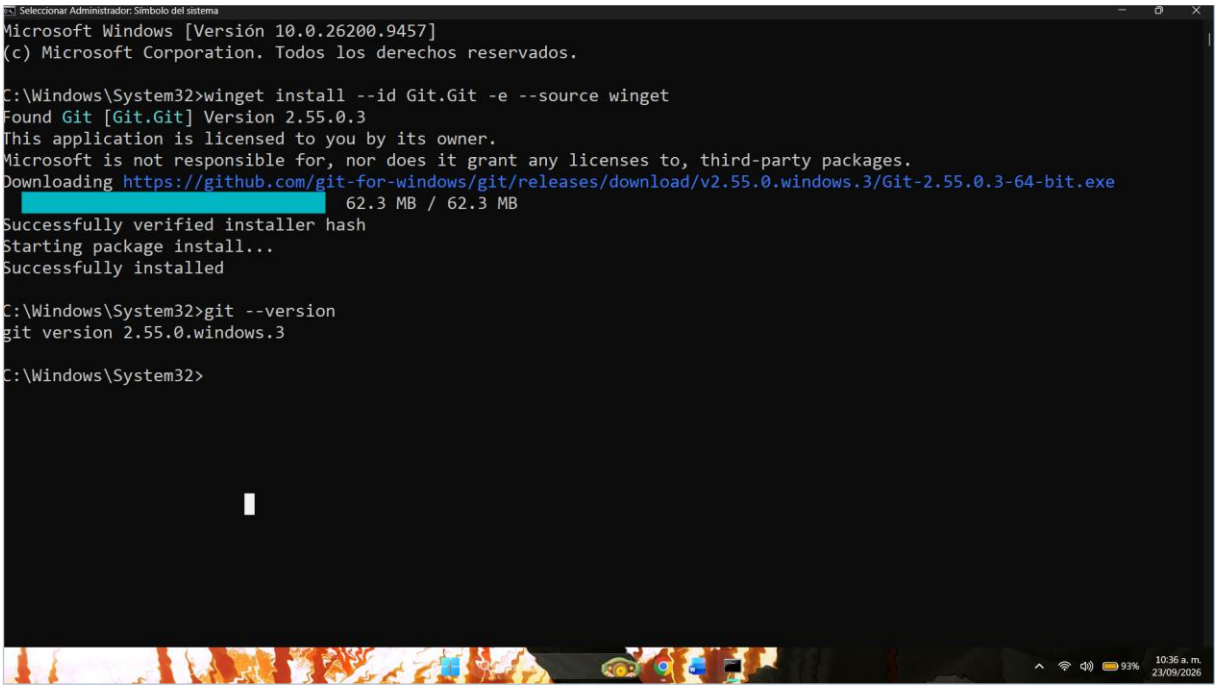


Figura 1. Instalación de Git y comprobación de la versión.

4. Node.js (LTS)

4.1 Descripción y requisitos

Node.js permite ejecutar JavaScript fuera del navegador y proporciona npm para administrar dependencias. La edición LTS se recomienda para entornos de desarrollo que priorizan estabilidad y soporte.

4.2 Instalación

```
winget install OpenJS.NodeJS.LTS
```

La evidencia registra Node.js (LTS) versión 24.19.0. Si Winget solicita aceptar los términos de la fuente Microsoft Store, revise el aviso y responda Y únicamente si está de acuerdo.

4.3 Verificación

```
node -v
npm -v
```

La primera orden debe mostrar “v24.19.0” según la captura. La segunda devuelve la versión de npm instalada con Node.js; su valor puede variar.

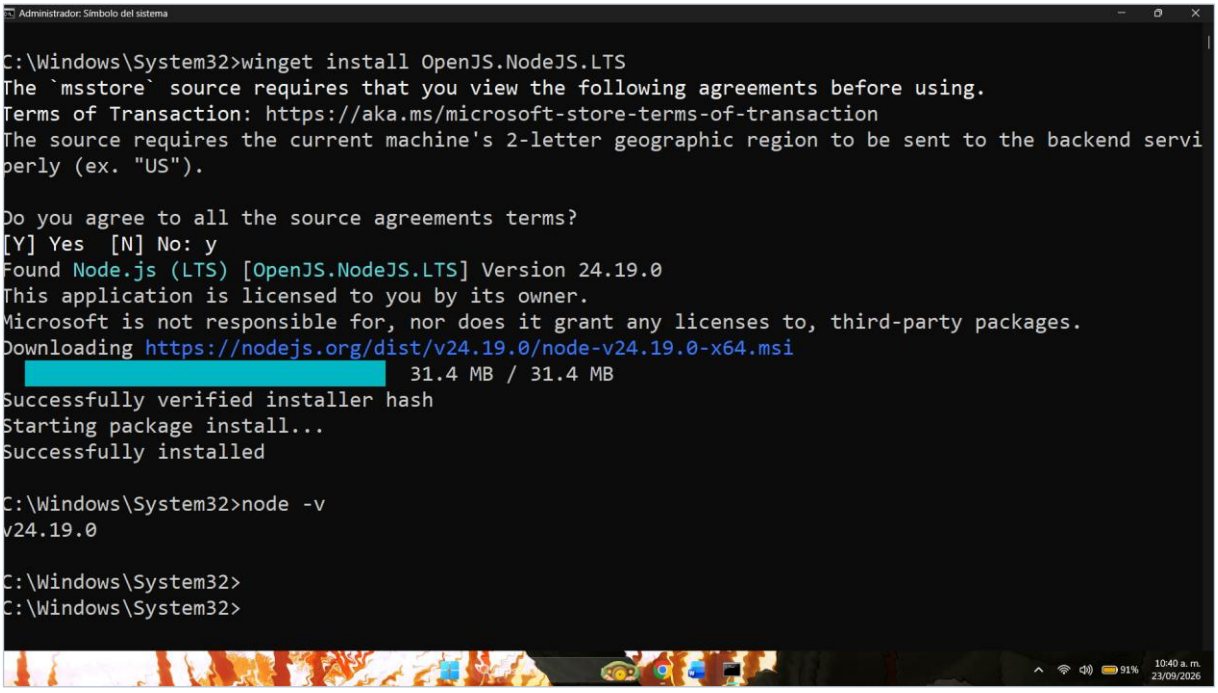


Figura 2. Instalación de Node.js LTS y salida de node -v.

5. Python

5.1 Descripción y requisitos

Python es un lenguaje de programación interpretado utilizado para automatización, scripts y herramientas de desarrollo. En este entorno se considera una dependencia útil para flujos de trabajo con herramientas como Codex y Archify.

5.2 Instalación

```
winget install Python.Python.3.13
```

La captura muestra la descarga e instalación de Python 3.13.15.

5.3 Verificación

```
py --version
python --version
```

La evidencia confirma “Python 3.13.15” mediante py --version. Si python no se reconoce, pruebe el lanzador py y revise que Python se encuentre en PATH.

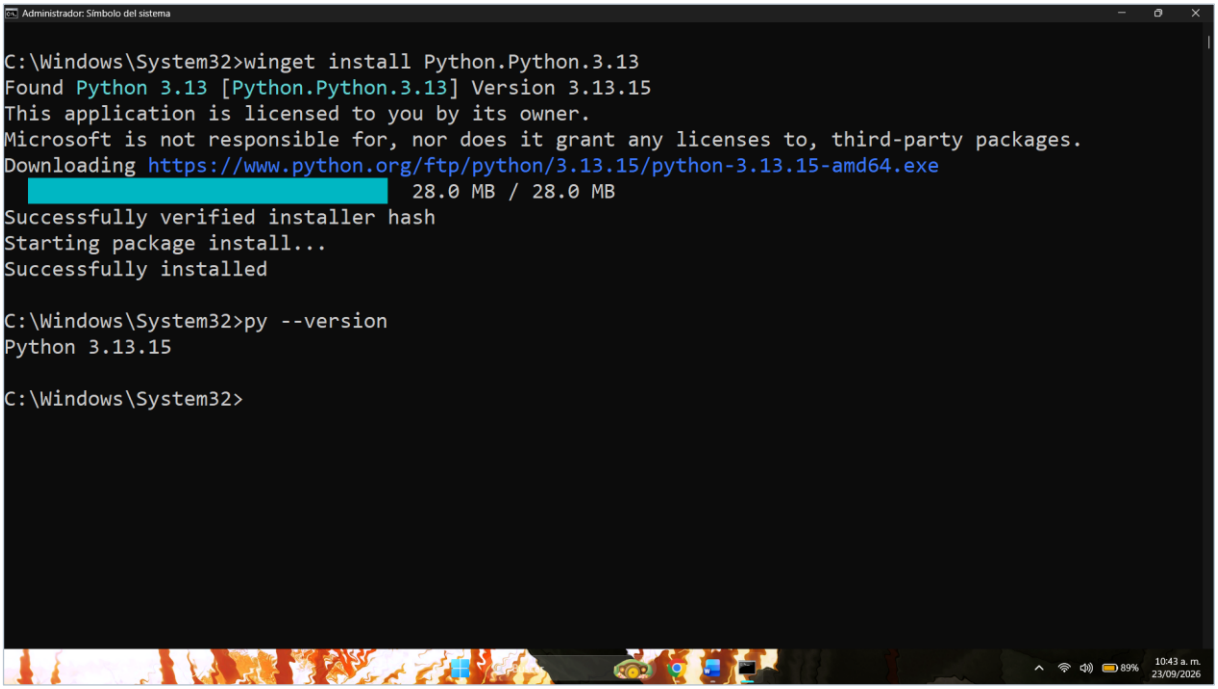


Figura 3. Instalación de Python 3.13 y verificación con py --version.

6. OpenAI Codex CLI

6.1 Descripción y requisitos

Codex CLI es una interfaz de línea de comandos para trabajar con proyectos de software desde una terminal. Antes de instalarlo, compruebe que Windows puede ejecutar PowerShell y que la conexión a Internet está disponible.

6.2 Instalación mediante PowerShell

```
powershell -ExecutionPolicy ByPass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
```

> El comando descarga y ejecuta el script de instalación publicado para Codex. `-ExecutionPolicy ByPass` se aplica a esa invocación de PowerShell; no equivale a cambiar permanentemente la política del sistema. Ejecute este procedimiento solo si confía en el origen del script.

La evidencia registra Codex CLI versión 0.156.1 y muestra que la ruta PATH se actualiza para futuras sesiones. Si la terminal actual no reconoce codex, ciérrela y abra una nueva.

6.3 Verificación y prueba interactiva

```
codex --version
codex
```

1. Ejecute `codex --version` para comprobar que el ejecutable está disponible.
2. Inicie `codex` desde la carpeta del proyecto, no desde una carpeta del sistema, para que el contexto de trabajo sea el adecuado.
3. Complete el inicio de sesión o autorización que solicite la herramienta.
4. Escriba una petición de prueba no destructiva, por ejemplo: `Resume la estructura de este proyecto sin modificar archivos.`

La captura muestra la interfaz interactiva iniciada y una respuesta de prueba. El acceso y las funciones disponibles dependen de la cuenta, la versión y los permisos configurados.

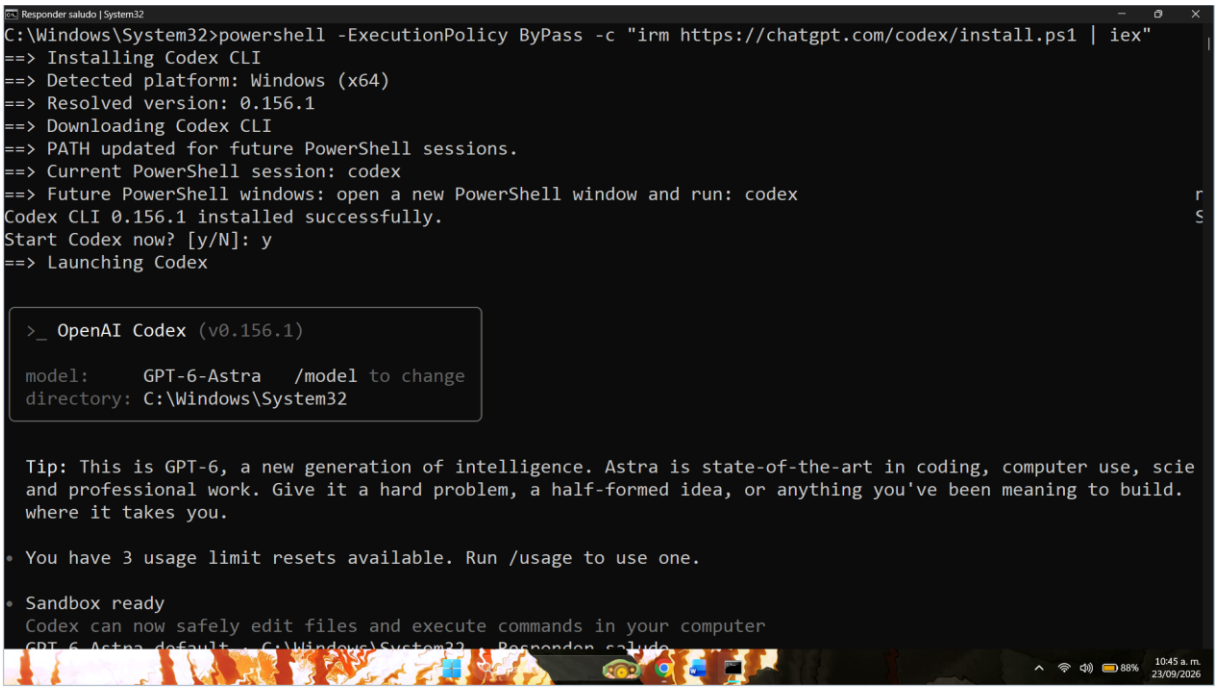


Figura 4. Instalación de Codex CLI e inicio de la interfaz interactiva.

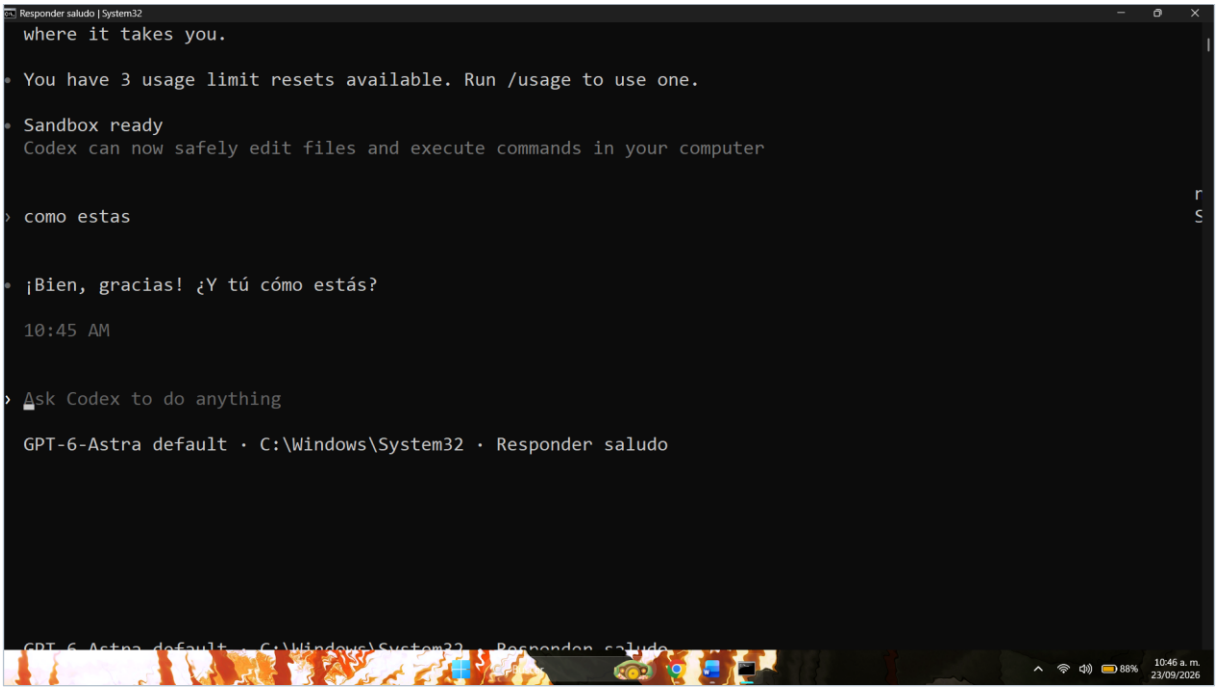


Figura 5. Prueba conversacional dentro de Codex CLI.

7. Archify: instalación de skills y agentes

7.1 Descripción y requisitos

Archify, en el flujo documentado, se incorpora mediante el gestor de skills ejecutado con npx. Las skills añaden instrucciones o capacidades reutilizables a agentes de programación compatibles. Se requiere Node.js/npm y acceso al repositorio remoto.

7.2 Instalación de skills

```
npx skills add tt-ali/archify -g
```

El indicador `-g` solicita una instalación global en el contexto admitido por el gestor. La evidencia detecta dos skills: `archify` y `archify-review`.

7.3 Selección de agentes y método

1. Cuando aparezca la selección de skills, marque `archify` y `archify-review` (o use Select All si desea instalar ambas). Confirme con Enter.
2. Seleccione los agentes de destino que utilice. La captura muestra opciones como Codex, Cursor, Cline y otros agentes; la lista puede cambiar según la versión.
3. Elija el método de instalación `Symlink (Recommended)` si es compatible con su entorno.
4. Revise el resumen, las rutas de instalación y los avisos de seguridad antes de aceptar.
5. Espere el mensaje de instalación completada. La captura también pregunta si se desea instalar la skill `find-skills`; es opcional.

7.4 Verificación

```
npx skills --help
```

Use la ayuda del gestor para revisar los comandos disponibles en la versión instalada. La evidencia visual es la confirmación principal del proceso: repositorio clonado, dos skills detectadas y pantalla de instalación completada.

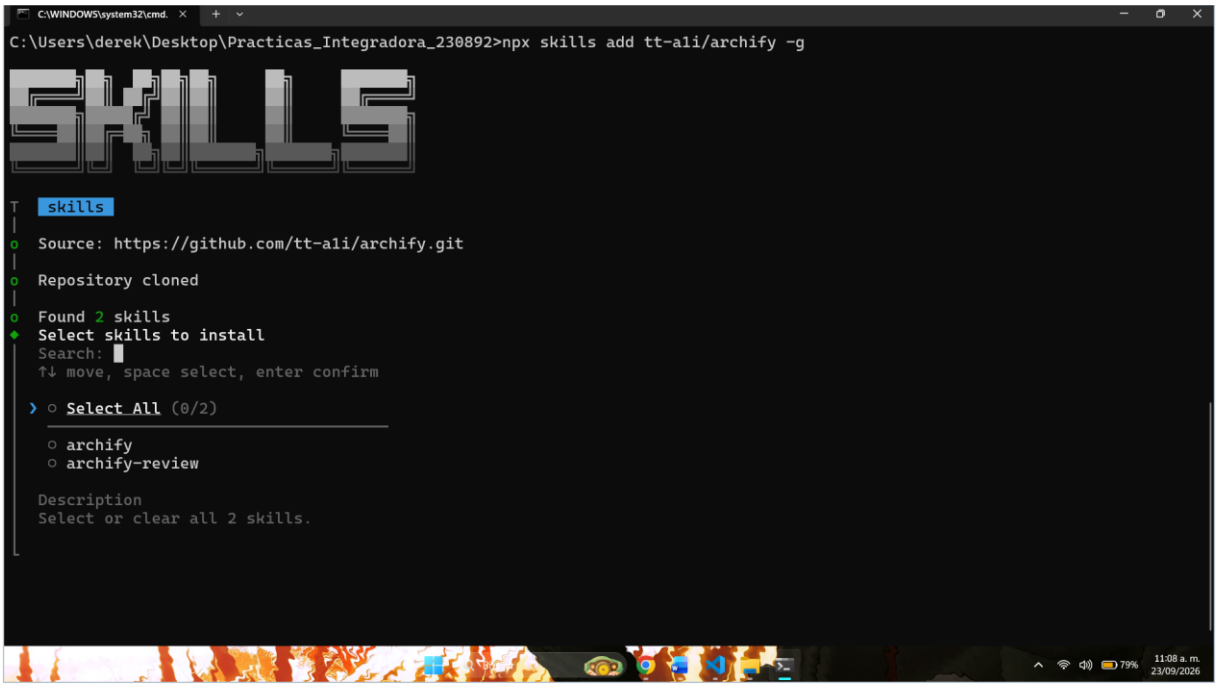


Figura 6. Detección de las skills de Archify.

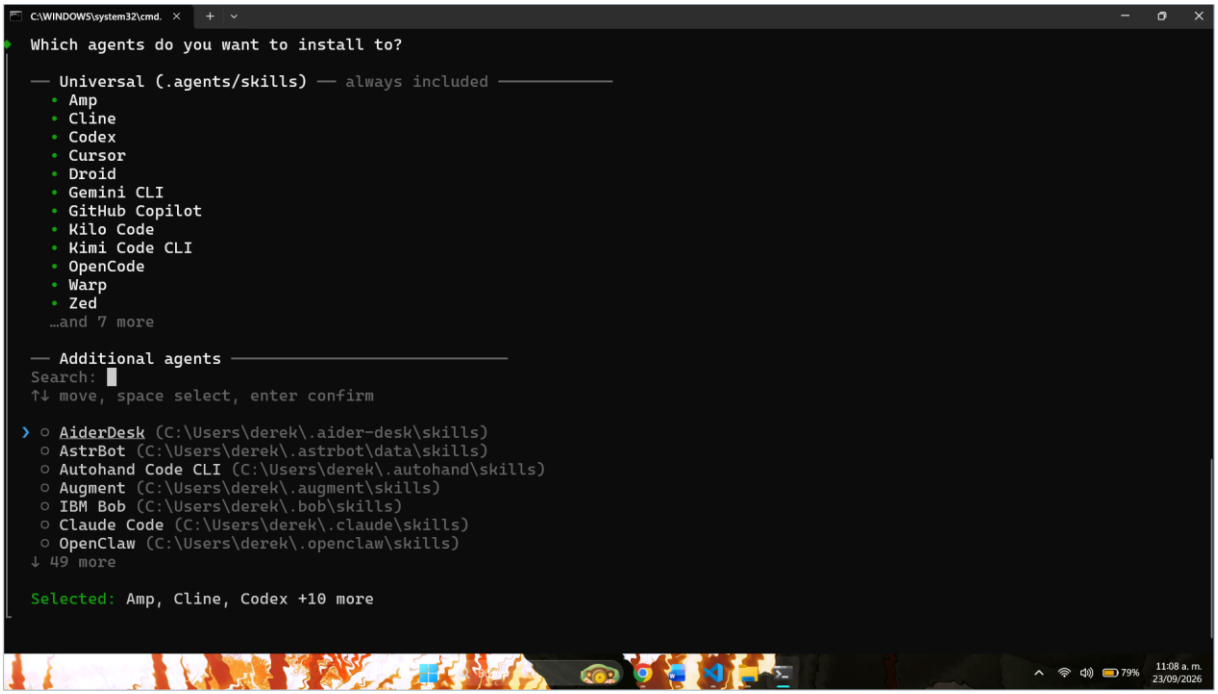


Figura 7. Selección de agentes compatibles.

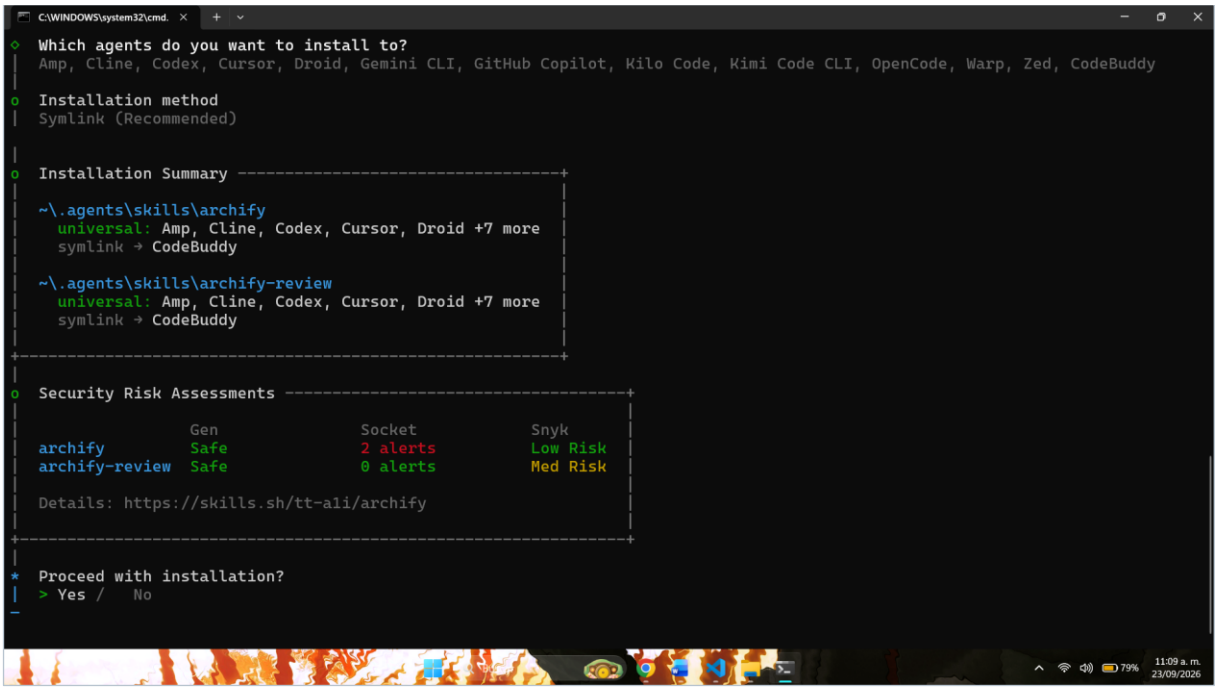


Figura 8. Resumen de instalación y alertas de seguridad mostradas por el gestor.

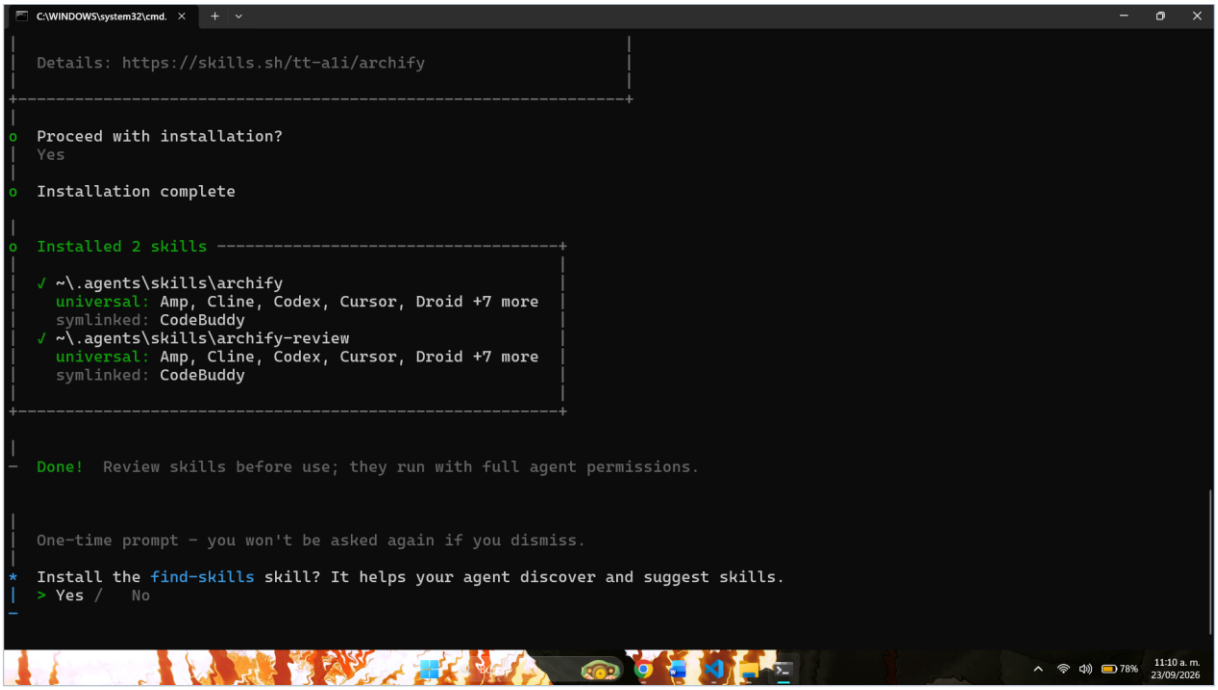


Figura 9. Confirmación de instalación completada.

8. Flutter SDK

8.1 Descripción y requisitos

Flutter es un SDK para crear aplicaciones multiplataforma a partir de una base de código común. La documentación oficial recomienda descargar el paquete estable, extraerlo en una ubicación controlada y añadir su carpeta `bin` a PATH.

8.2 Descarga y extracción

1. Abra la documentación oficial de instalación de Flutter: https://docs.flutter.dev/install.
2. Descargue el paquete estable de Windows indicado para este procedimiento: `flutter_windows_3.47.5-stable.zip`.
3. Extraiga el contenido en una ruta sin permisos restrictivos, por ejemplo `C:\src\flutter`. Evite rutas que requieran permisos elevados o que contengan caracteres problemáticos.
4. Agregue la carpeta `bin` del SDK a la variable de entorno PATH del usuario. Ejemplo: `C:\src\flutter\bin` si esa fue la ruta elegida.

8.3 Verificación

```
flutter --version
flutter doctor
```

`flutter --version` debe mostrar la versión instalada. `flutter doctor` analiza dependencias y herramientas complementarias e indica qué componentes necesitan configuración. El resultado depende de los objetivos de desarrollo (Android, web, Windows, etc.).

> **Nota:** la captura de referencia muestra la documentación oficial y el procedimiento general de descarga. El nombre del archivo ZIP y la versión 3.47.5 se documentan conforme a los datos proporcionados para esta guía; verifique en el archivo descargado que la versión y el canal coincidan.

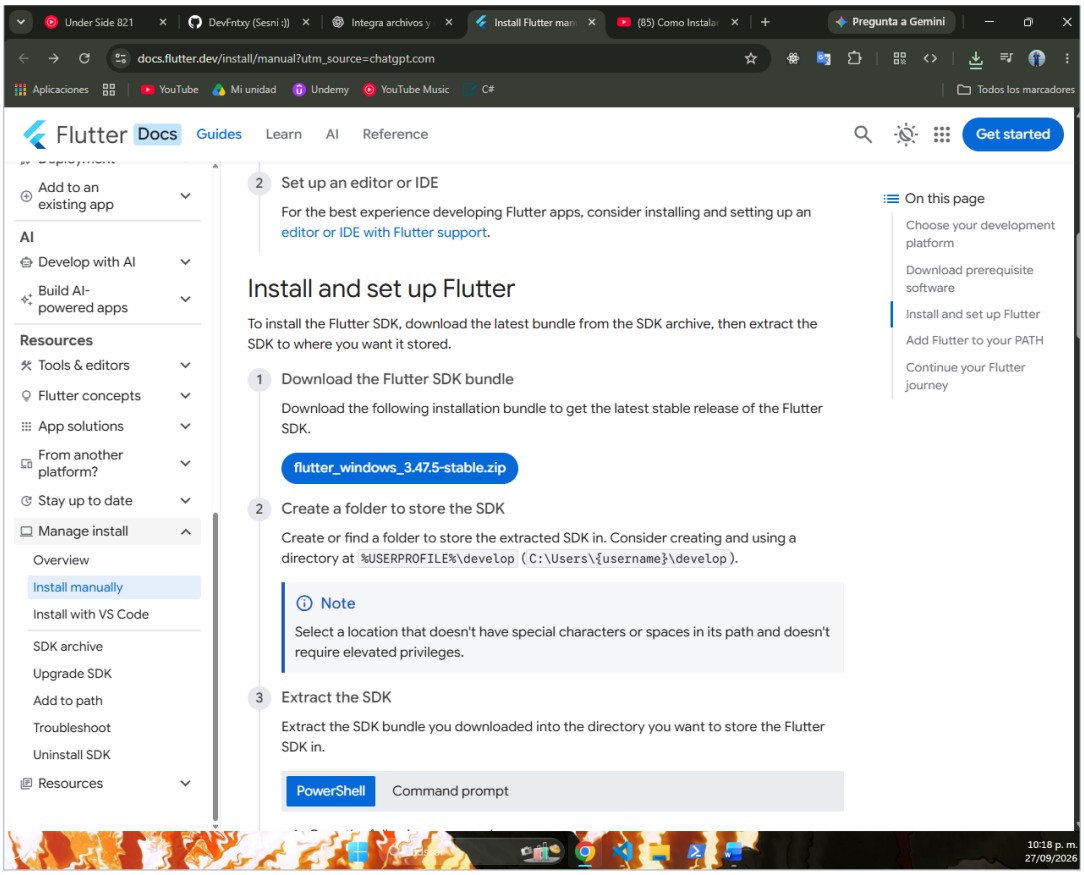


Figura 10. Página oficial de Flutter con instrucciones para descargar el SDK.

9. Docker Desktop y WSL

9.1 Descripción y requisitos

Docker Desktop proporciona una interfaz y un motor para crear y ejecutar contenedores. En Windows, su backend basado en WSL 2 permite ejecutar componentes Linux y contenedores Linux sin administrar manualmente una máquina virtual tradicional.

La evidencia muestra Docker Desktop 4.92.0 (240144) y posteriormente un aviso que indica que WSL no está instalado. El proceso se resolvió instalando WSL y la distribución Ubuntu.

9.2 Instalación de Docker Desktop

1. Acceda al sitio oficial: https://www.docker.com/products/docker-desktop/.
2. Descargue el instalador de Windows correspondiente a la arquitectura del equipo.
3. Ejecute el instalador y seleccione la opción recomendada para el usuario actual, salvo que los requisitos administrativos indiquen otra modalidad.
4. Acepte los términos aplicables y complete la instalación. Reinicie el equipo si el instalador o Windows lo solicita.

La evidencia registra “Docker Desktop 4.92.0 — Installation succeeded”. La disponibilidad de una versión concreta depende de la fecha y del canal de distribución.

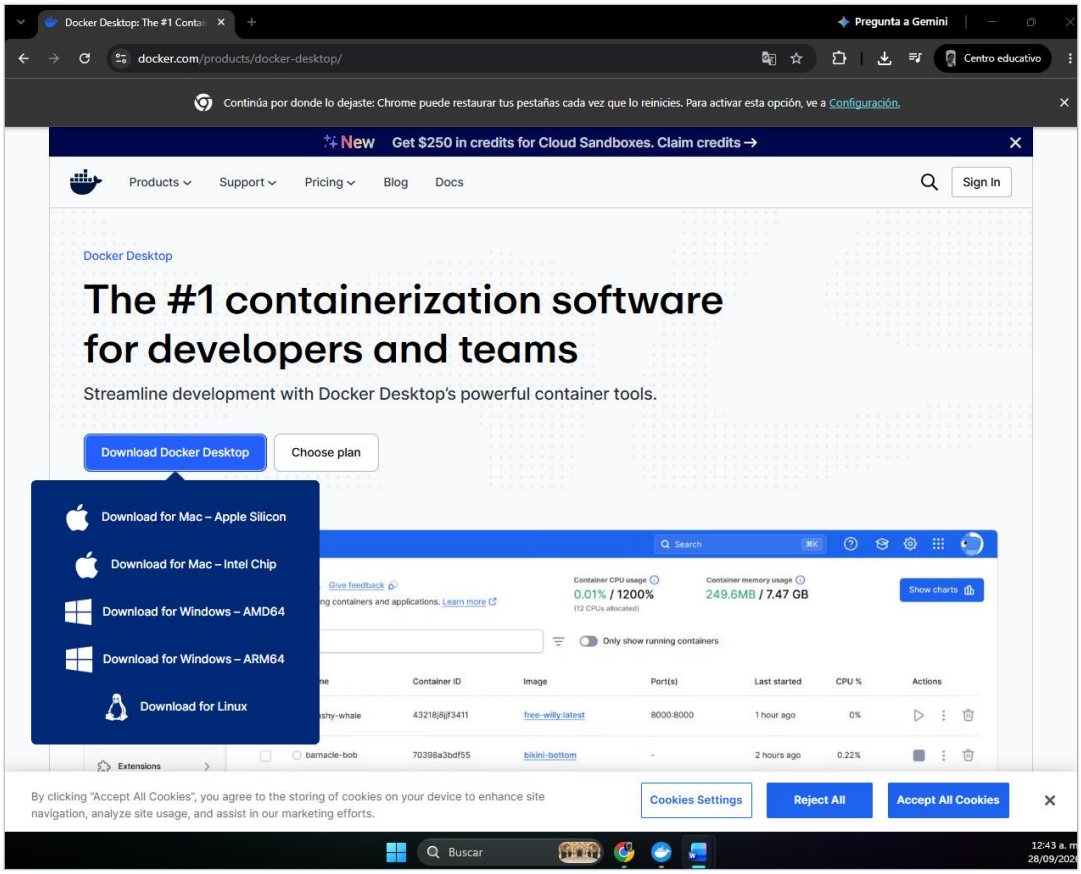


Figura 11. Página de descarga de Docker Desktop.

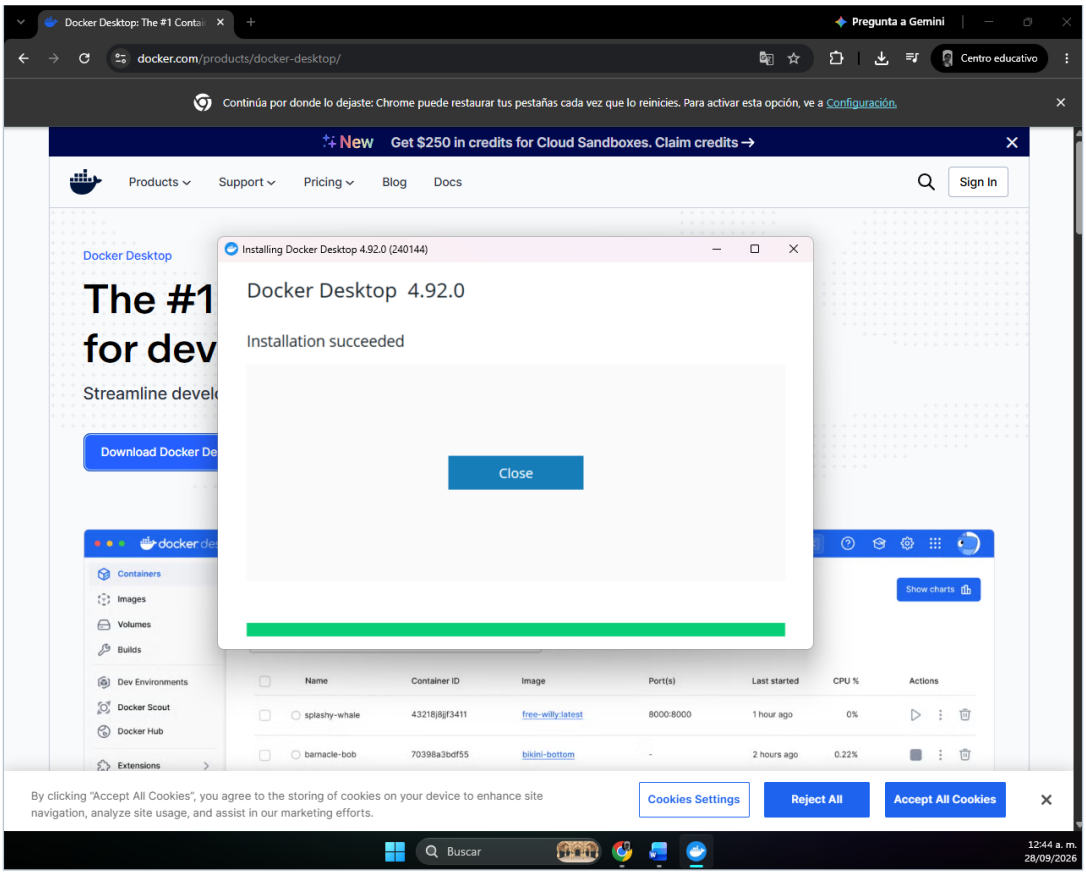


Figura 12. Mensaje de instalación correcta de Docker Desktop 4.92.0.

9.3 Activar características de Windows

Abra el cuadro Ejecutar con Win + R, escriba `optionalfeatures.exe` y presione Enter. En la ventana "Características de Windows", habilite las opciones siguientes cuando estén disponibles:

- Plataforma de máquina virtual.
- Subsistema de Windows para Linux.
- Plataforma del hipervisor de Windows.
- Hyper-V y sus componentes, si la edición de Windows los ofrece y el escenario lo requiere.

Aplique los cambios y reinicie Windows si se solicita. Hyper-V no aparece en todas las ediciones de Windows; su ausencia no significa necesariamente que WSL 2 no pueda funcionar.

9.4 Resolver el error "WSL is not installed"

Al abrir Docker Desktop, la captura muestra un aviso de que WSL no está instalado. Para instalar el componente, abra PowerShell como administrador y ejecute:

```
wsl --install
```

El comando instala WSL y, de forma predeterminada, puede instalar una distribución Linux como Ubuntu. Espere a que terminen las descargas e instalaciones. Reinicie el equipo si se solicita.

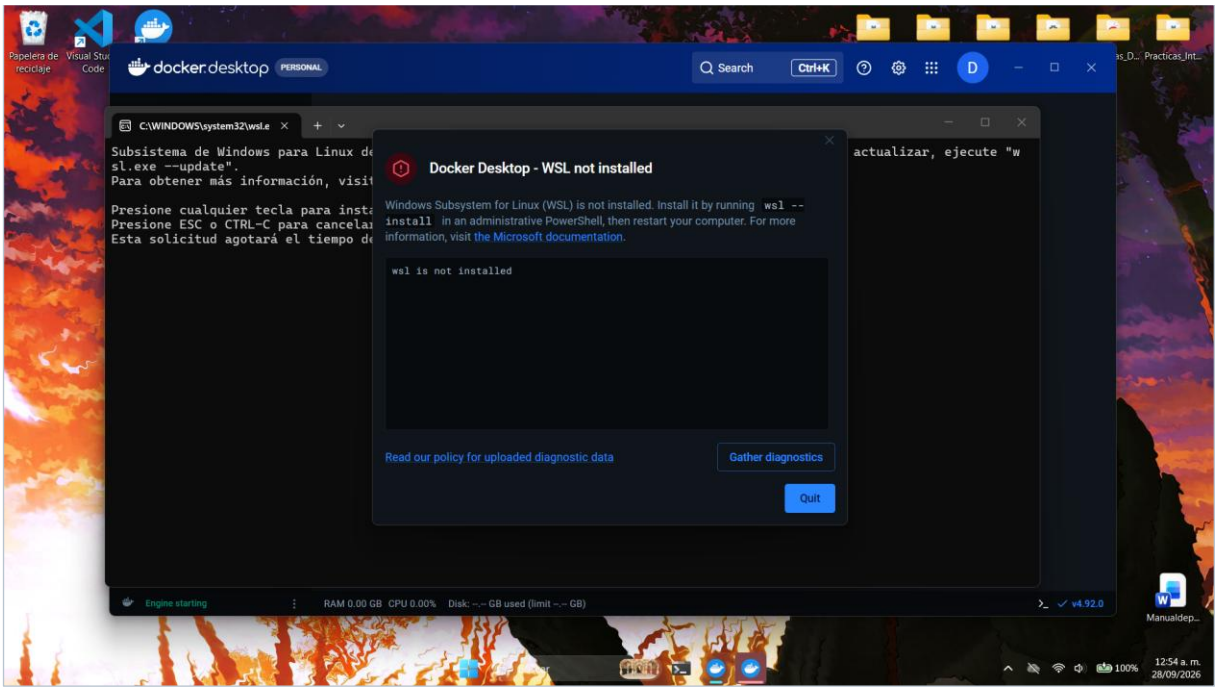


Figura 13. Aviso de Docker Desktop que identifica que WSL no está instalado.

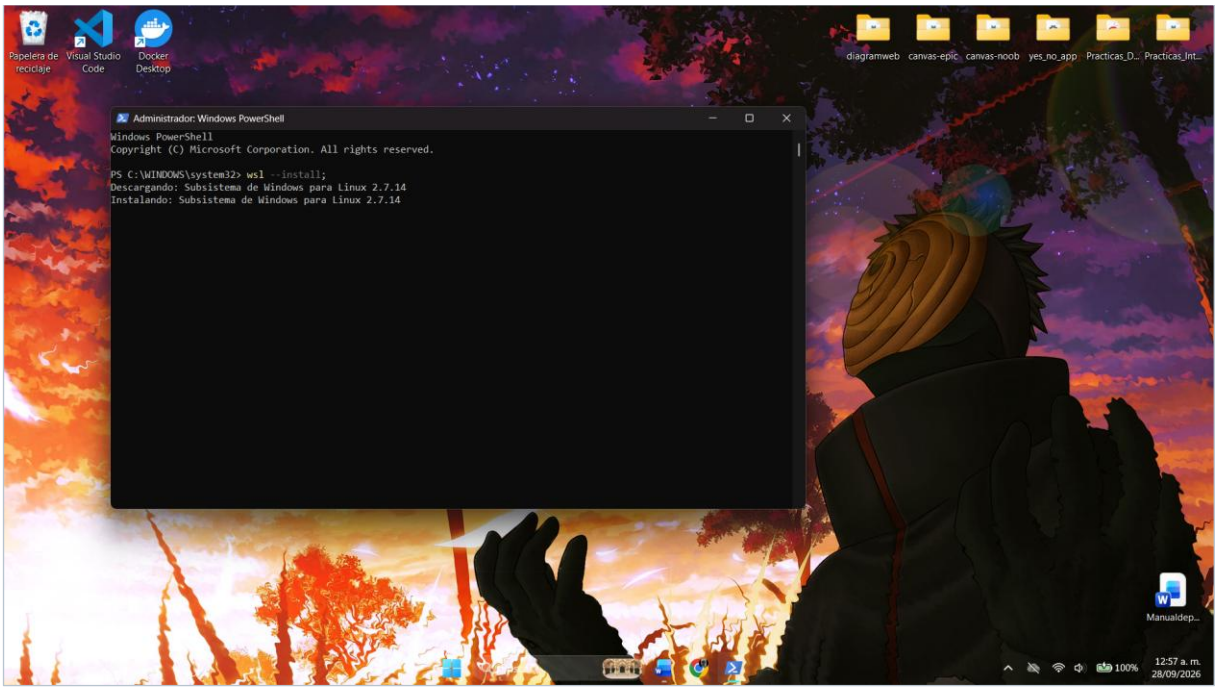


Figura 14. Instalación de WSL desde PowerShell mediante wsl --install.

9.5 Configuración inicial de Ubuntu

1. Después de instalarse, Ubuntu se inicia y solicita crear una cuenta Unix local.
2. Introduzca un nombre de usuario Linux. La evidencia utiliza `derek`; puede elegirse otro nombre.
3. Escriba una contraseña y repítala cuando se solicite. Mientras se escribe, Linux normalmente no muestra caracteres en pantalla; esto es esperado.

4. Responda a la pregunta opcional de telemetría con `y` o `n`, según su preferencia. Escriba una sola respuesta válida y presione Enter.
5. Espere a que aparezca el prompt de Ubuntu, por ejemplo `derek@...:...$`. Esto indica que la sesión de Linux está lista.

> La contraseña de Ubuntu es independiente de la contraseña de Windows y no se muestra en la terminal. No la incluya en capturas ni la comparta.

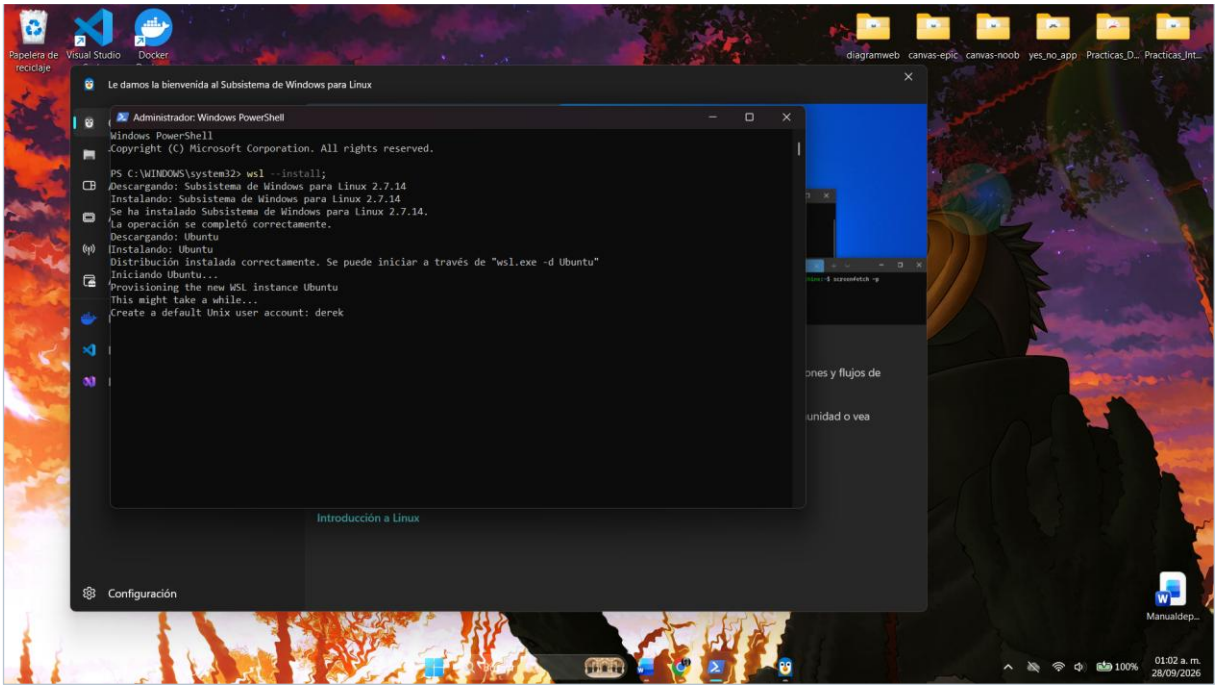


Figura 15. Instalación de Ubuntu e inicio de la creación del usuario Linux.

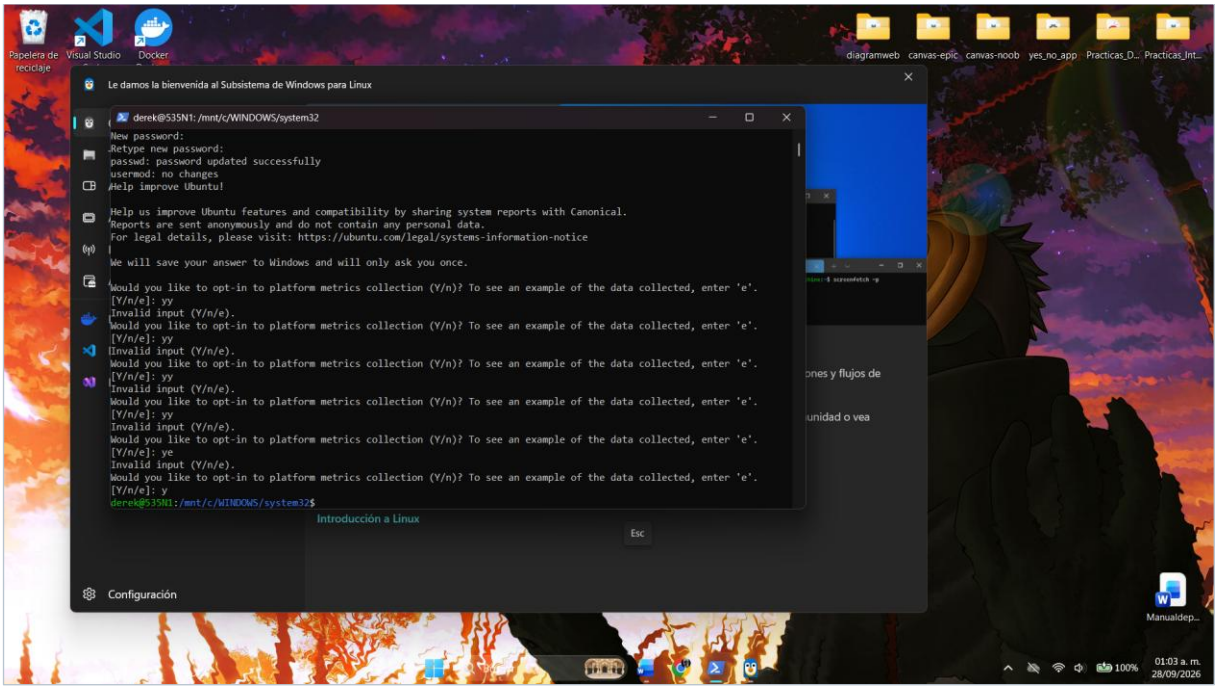


Figura 16. Configuración de contraseña y pregunta de métricas de Ubuntu.

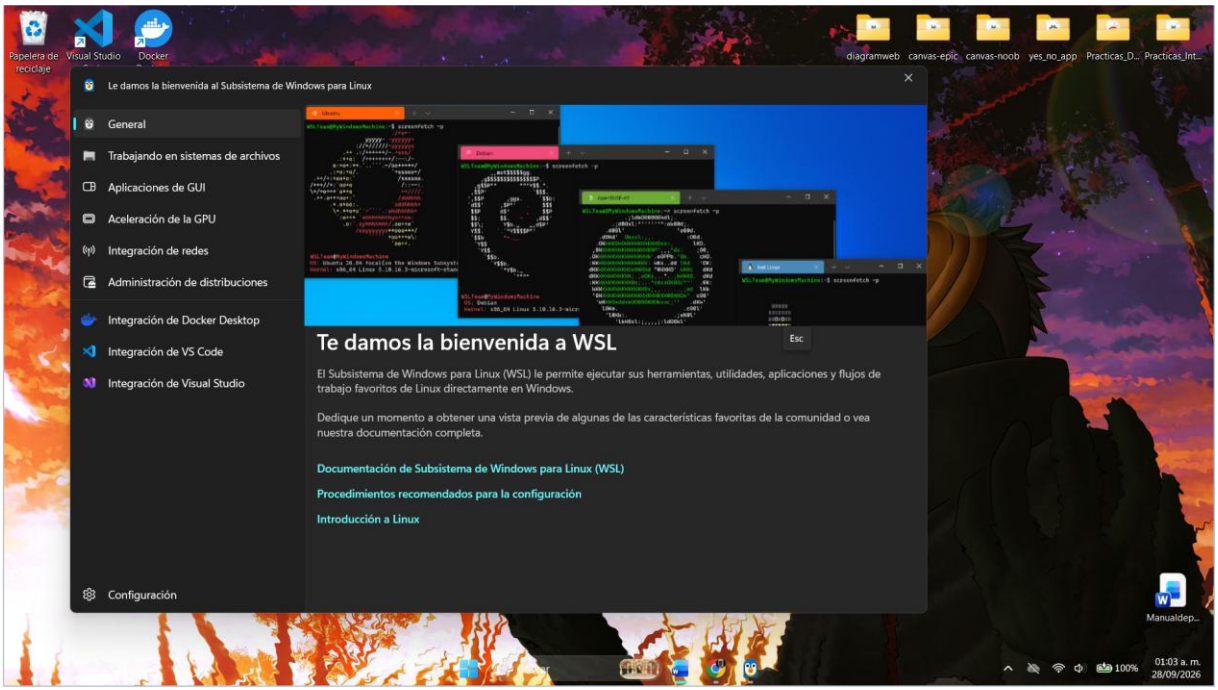


Figura 17. Interfaz de bienvenida y documentación de WSL.

9.6 Verificación de WSL y Docker

```
wsl --status
wsl --list --verbose
```

Compruebe que Ubuntu aparece en la lista y que la versión de WSL es compatible con el backend requerido. Para establecer WSL 2 como versión predeterminada cuando proceda, ejecute:

```
wsl --set-default-version 2
```

Abra Docker Desktop y espere a que el motor indique que está en ejecución. Después, desde CMD o PowerShell, ejecute:

```
docker --version
docker info
docker run hello-world
```

Resultados esperados: `docker --version` muestra la versión del cliente; `docker info` presenta información del motor sin error de conexión; `docker run hello-world` descarga la imagen de prueba si hace falta y muestra un mensaje de ejecución satisfactoria. Si `docker` no se reconoce, abra una terminal nueva o revise la instalación.

9.7 Integración de Docker Desktop con WSL

1. En Docker Desktop, abra Settings y localice la configuración de WSL 2 o la integración con distribuciones Linux.
2. Active el motor basado en WSL 2 y, si está disponible, habilite la integración con Ubuntu.
3. Aplique los cambios y espere a que Docker Desktop reinicie el motor.

4. Desde Ubuntu, compruebe la disponibilidad de Docker según la configuración de integración aplicada. Si la distribución no aparece, revise que WSL y Docker Desktop estén actualizados y que Ubuntu esté detenido/iniciado correctamente.

10. Dockerizar una aplicación

Con Docker Desktop y WSL ya instalados y verificados (capítulo 9), este capítulo explica los dos archivos necesarios para dockerizar una aplicación y muestra, mediante capturas, la compilación de un APK móvil con Docker y su instalación en un teléfono. Los ejemplos corresponden al proyecto `hello_world_app`.

10.1 Archivos Dockerfile y .dockerignore

Una vez que la aplicación a dockerizar funciona sin errores, se crean dos archivos importantes: `Dockerfile` y `.dockerignore`.

Un **Dockerfile** es un archivo de texto plano que contiene una lista de instrucciones paso a paso para crear una imagen de Docker. Funciona como una «receta de cocina»: le indica a Docker qué sistema operativo base utilizar, qué dependencias instalar, qué archivos copiar y qué comando ejecutar al arrancar el contenedor.

El archivo **.dockerignore** es un archivo de texto que se coloca en la raíz de un proyecto para indicar a Docker qué archivos y carpetas debe ignorar al crear una imagen. Funciona de forma muy similar al archivo .gitignore de Git.

10.2 Ejemplo de .dockerignore

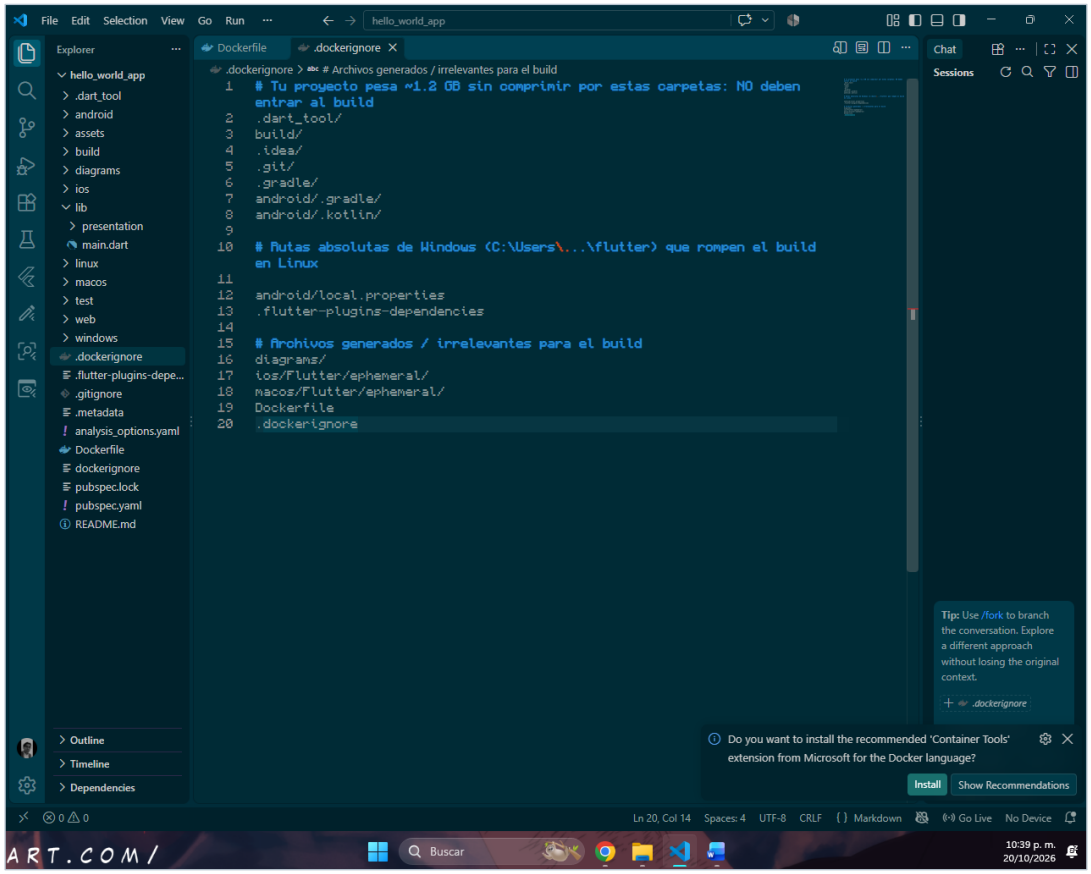


Figura 18. Ejemplo de archivo .dockerignore.

10.3 Ejemplo de Dockerfile

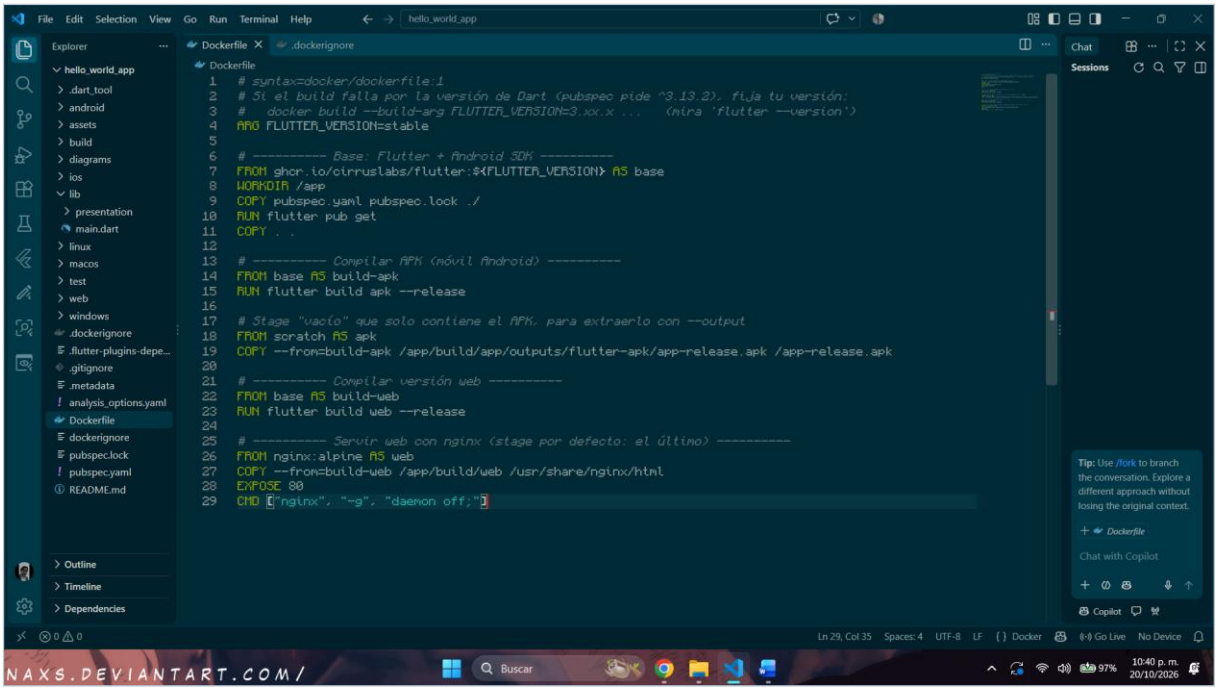


Figura 19. Ejemplo de Dockerfile.

10.4 Compilación del APK (móvil)

Desde el Símbolo del sistema, ubíquese en la carpeta del proyecto y ejecute el siguiente comando. El punto final indica que el contexto de compilación es la carpeta actual; si se omite, Docker responde con el error mostrado en la Figura 21.

```
docker build --target apk --output out .
```

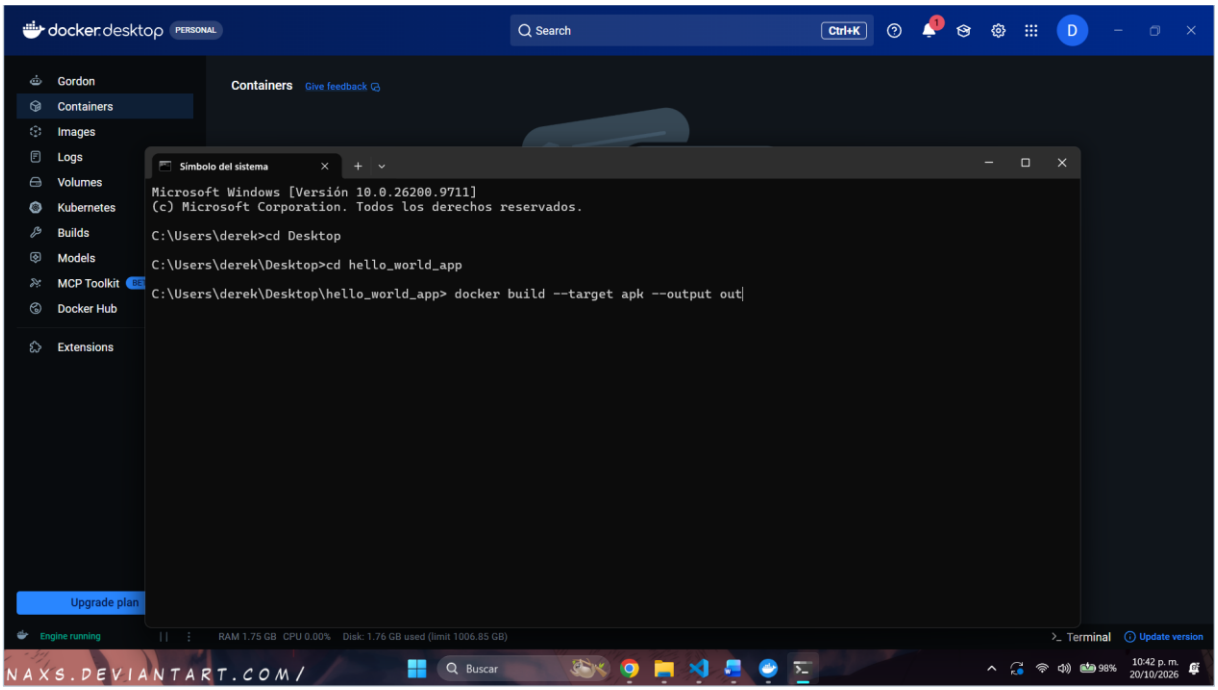


Figura 20. Símbolo del sistema en la carpeta hello_world_app con los comandos cd y docker build --target apk --output out.

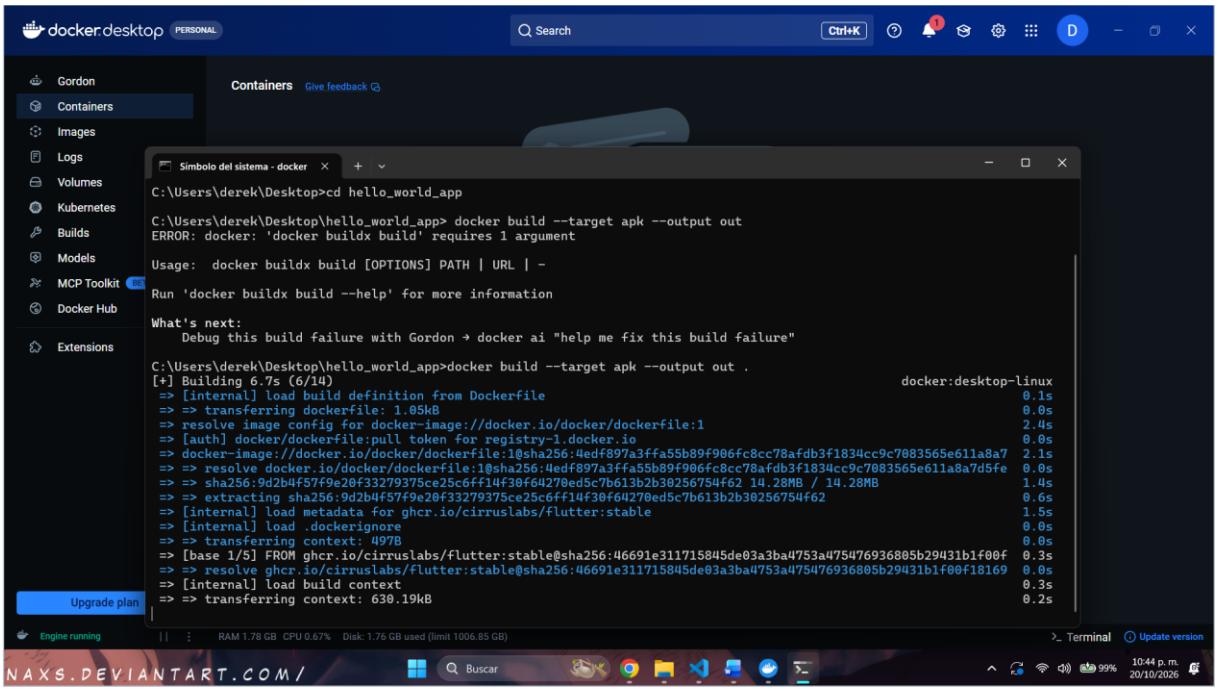


Figura 21. Error de Docker al omitir la ruta del contexto de compilación («requires 1 argument»).

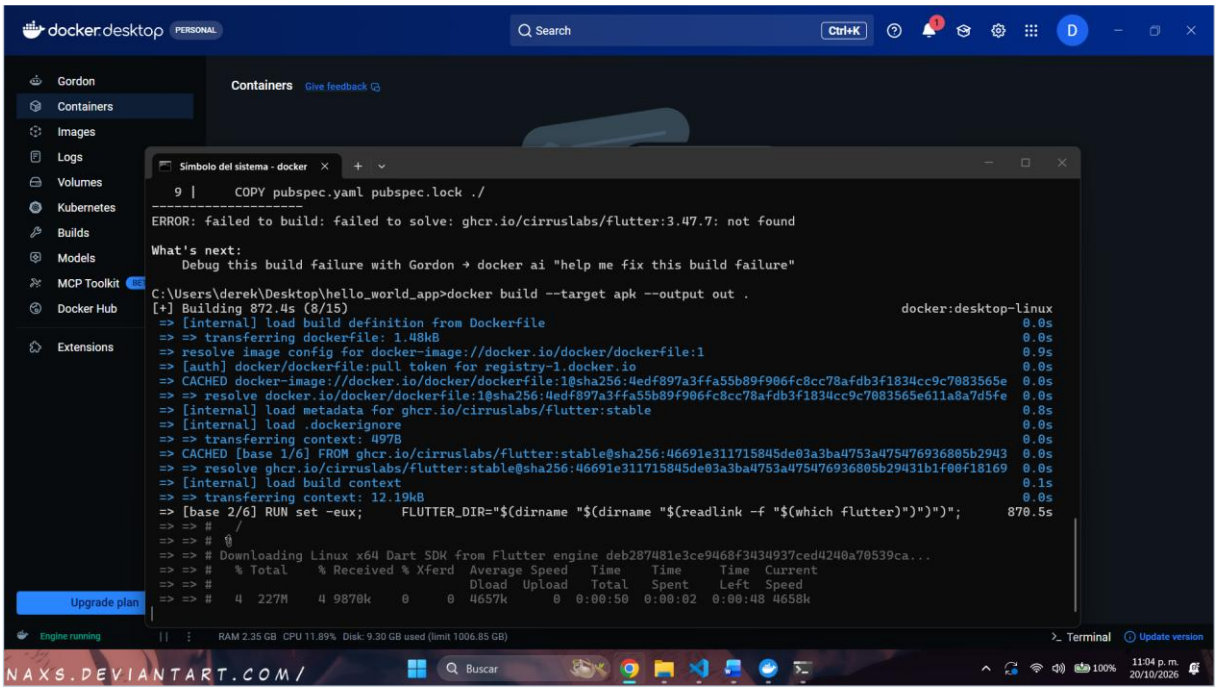


Figura 22. Compilación del APK con docker build --target apk --output out . (avance, parte 1 de 3).

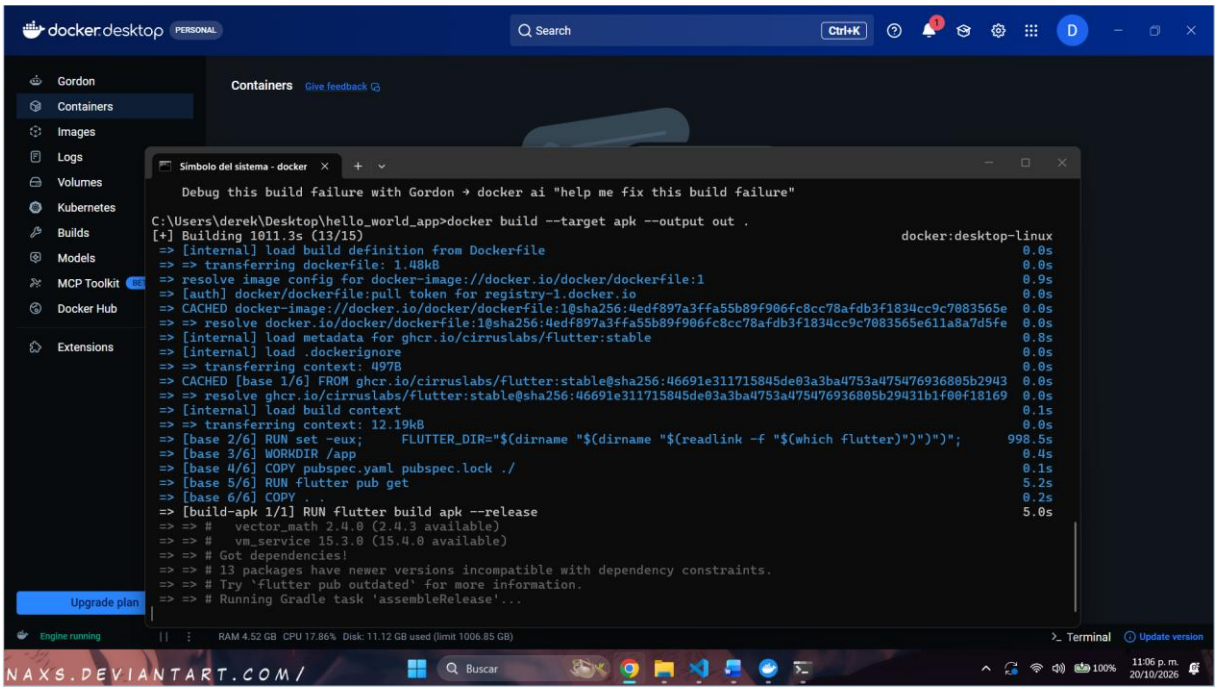


Figura 23. Compilación del APK (avance, parte 2 de 3).

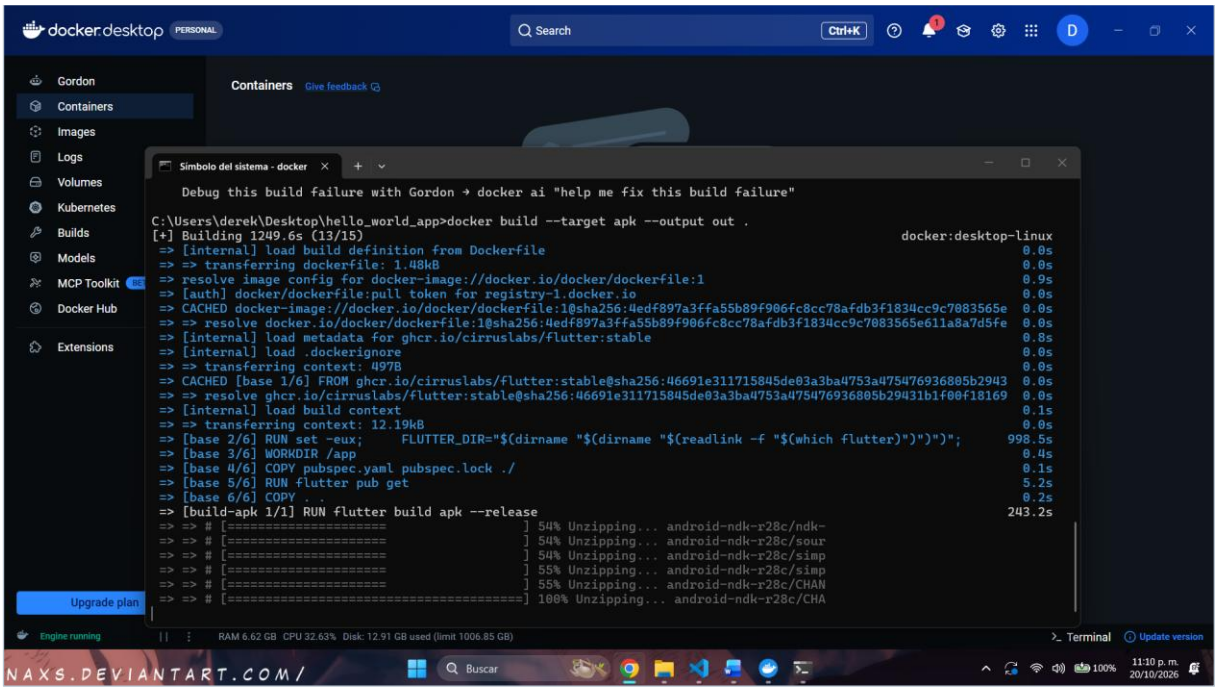


Figura 24. Compilación del APK (avance, parte 3 de 3).

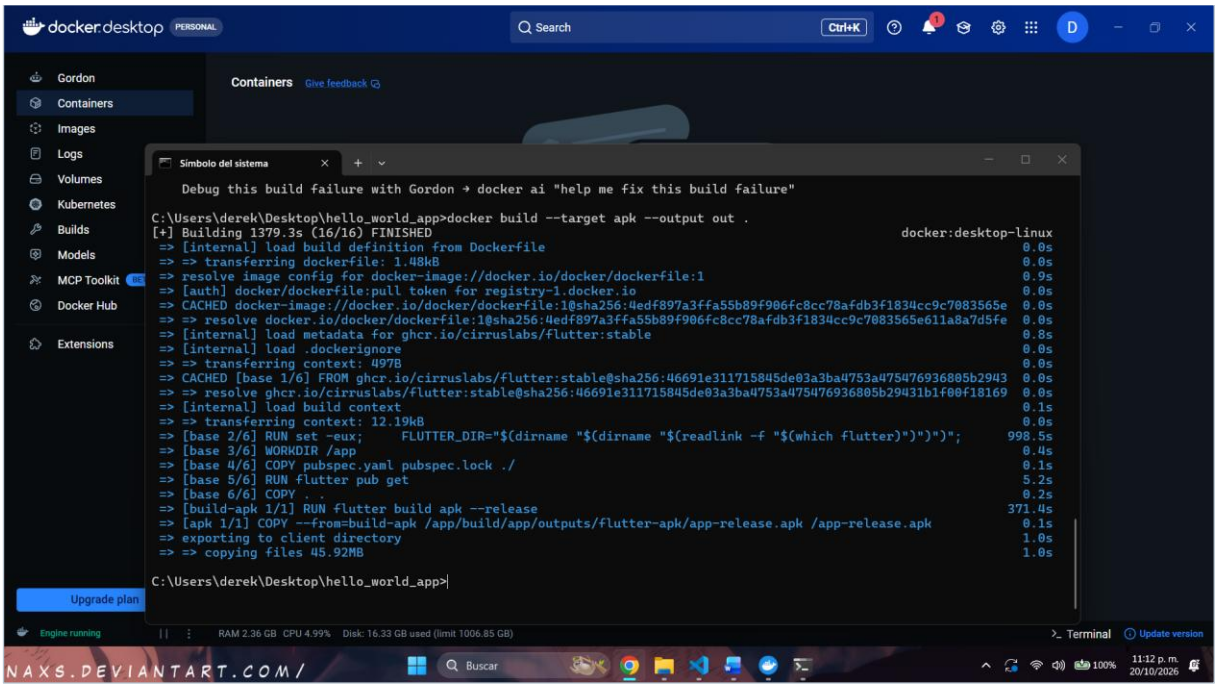


Figura 25. Compilación finalizada correctamente (16/16 FINISHED).

Una vez finalizada la compilación, el archivo `app-release.apk` se envía al teléfono (Figuras 26 y 27) y se instala siguiendo los avisos de Android (Figuras 28 a 30). Al terminar, la aplicación se ejecuta en el dispositivo (Figura 31).

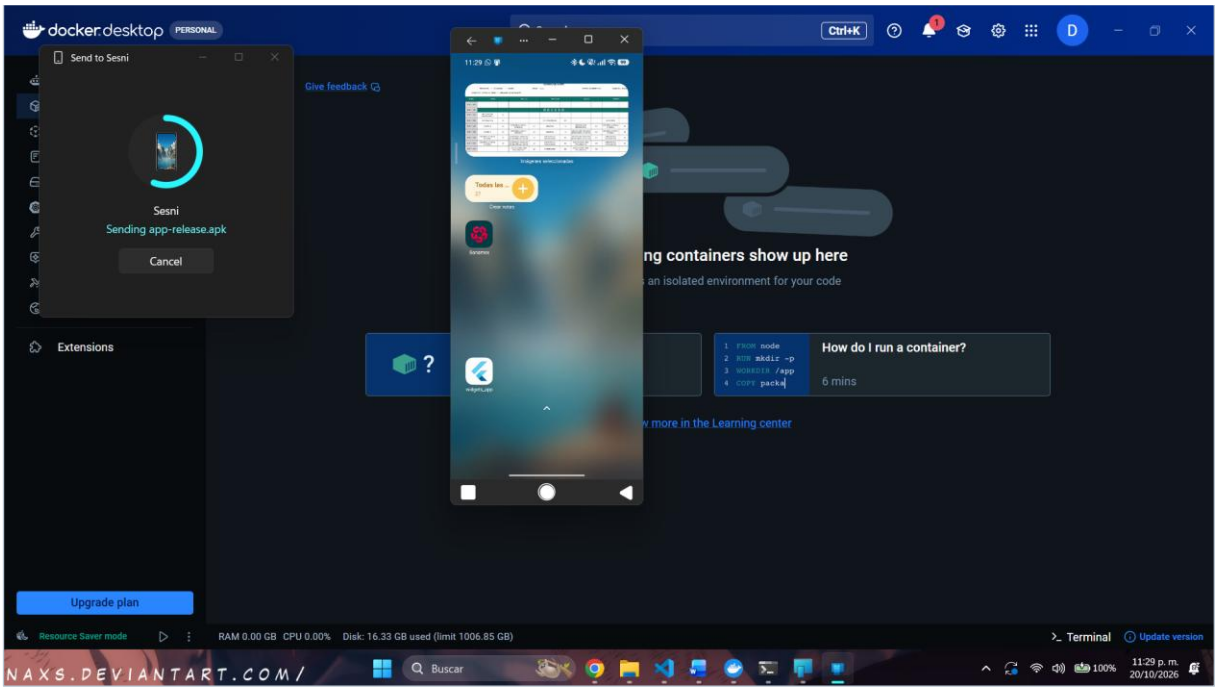


Figura 26. Envío del archivo app-release.apk al teléfono con Send to Sesni.

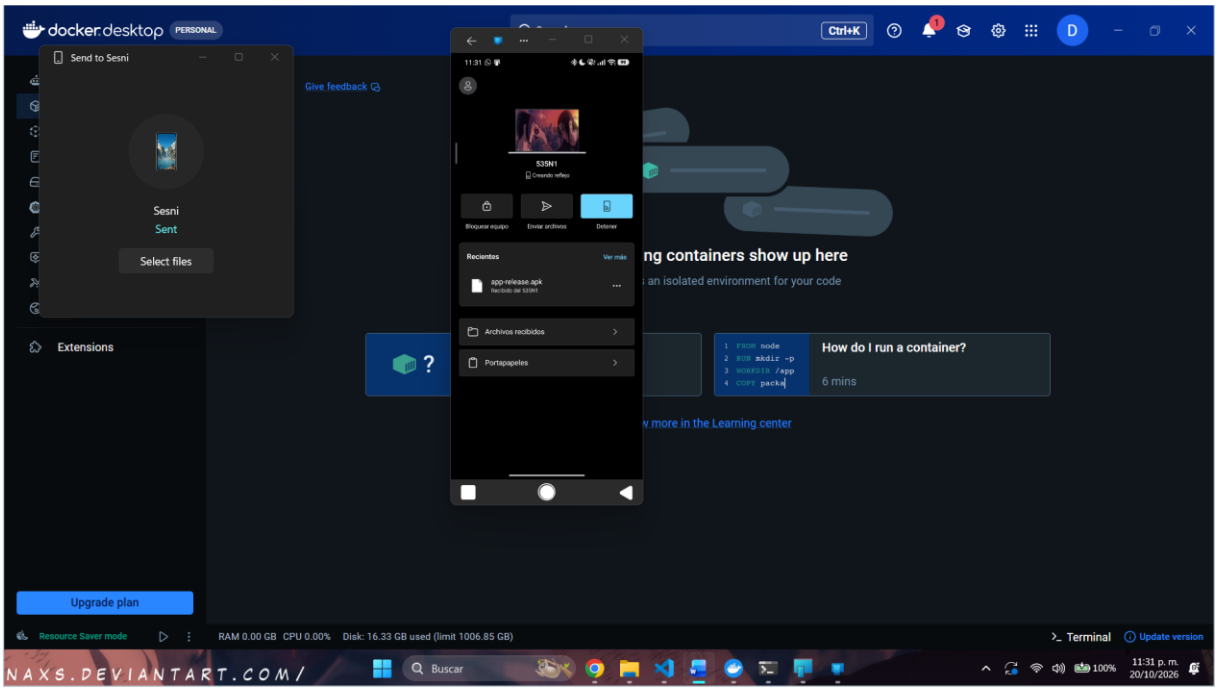


Figura 27. Archivo app-release.apk recibido en el teléfono.

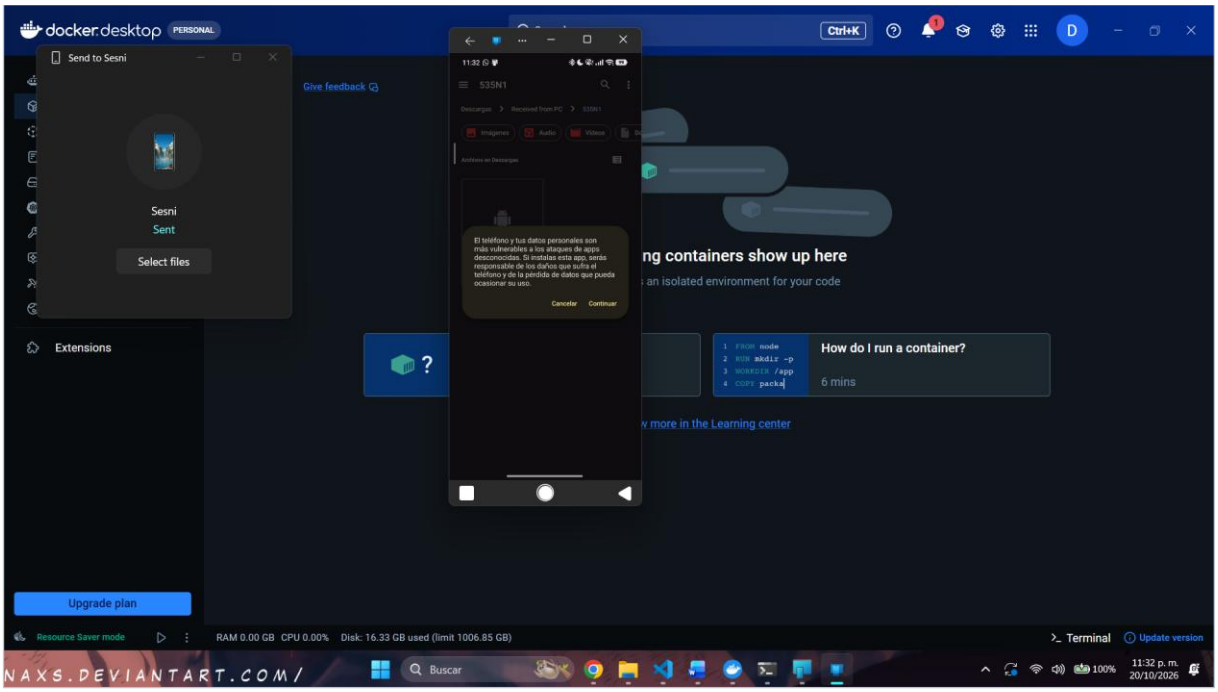


Figura 28. Aviso de seguridad de Android sobre aplicaciones desconocidas.

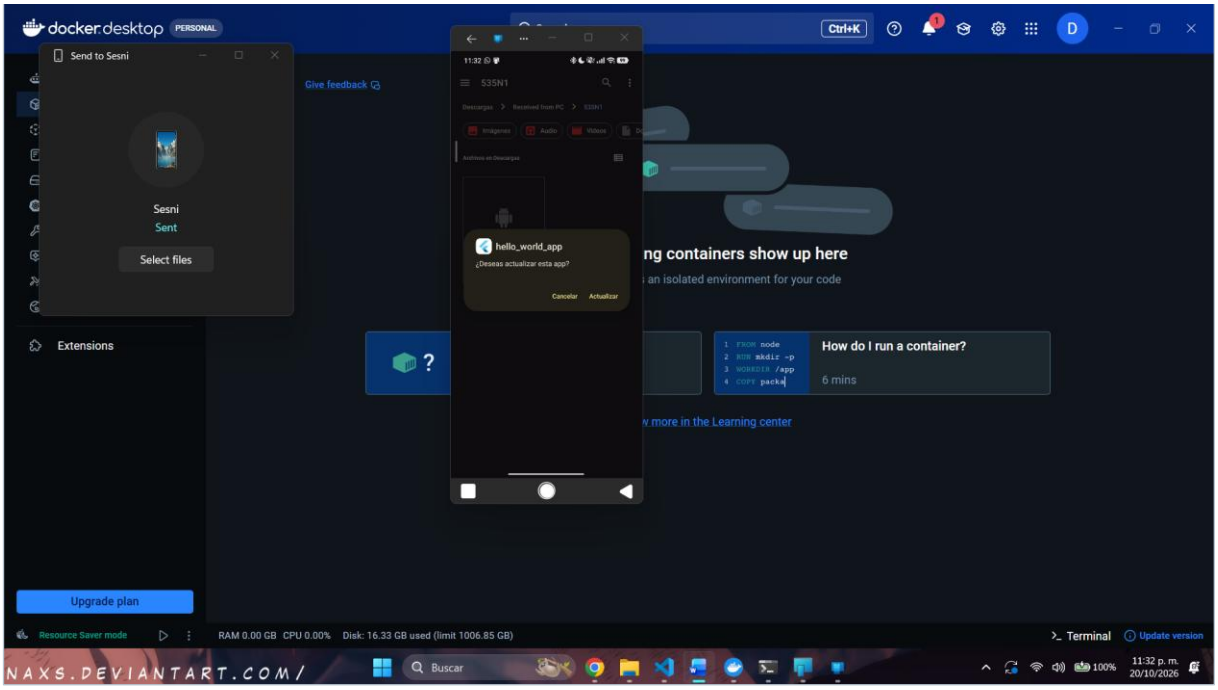


Figura 29. Confirmación de Android para actualizar hello_world_app.

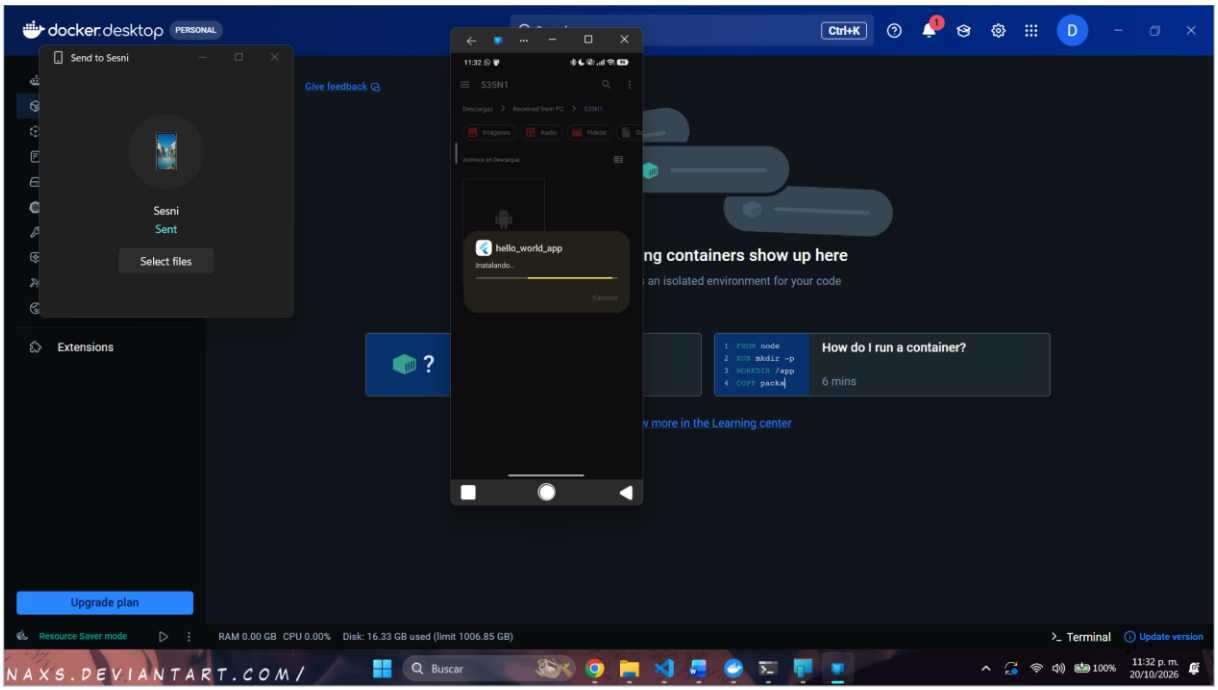


Figura 30. Instalación de hello_world_app en curso.

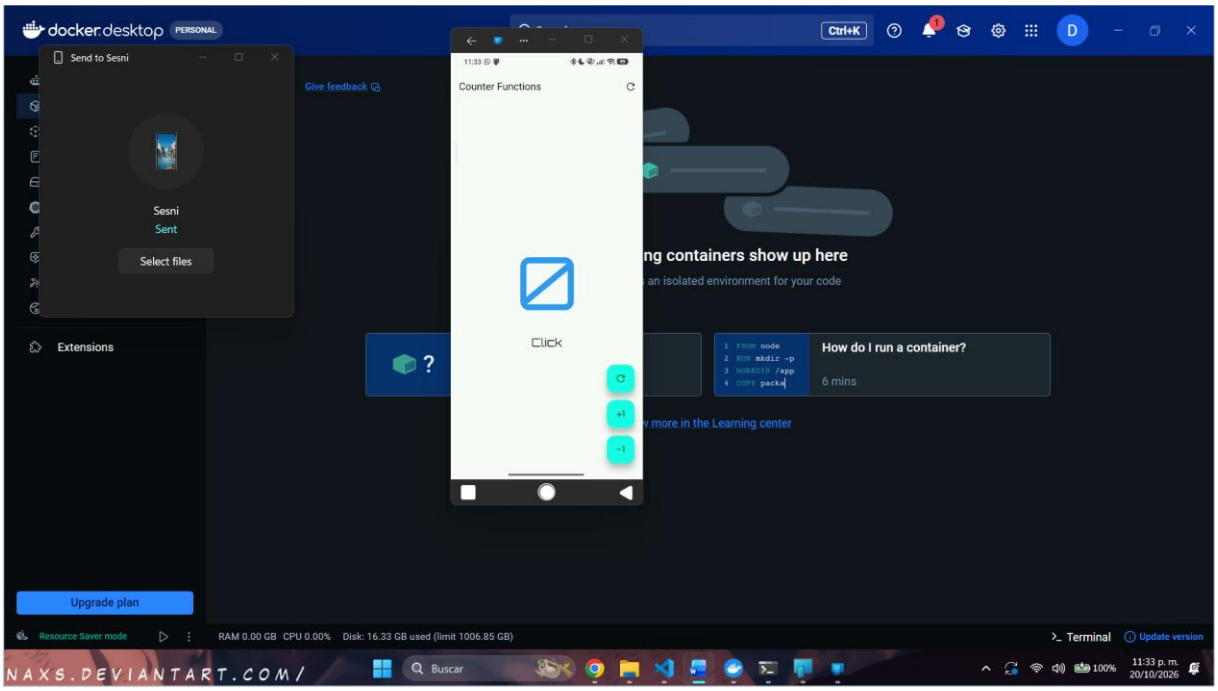


Figura 31. Aplicación hello_world_app en ejecución en el teléfono.

11. Verificación integral

Ejecute las siguientes comprobaciones desde una terminal nueva de Windows. Las versiones deben ser coherentes con las instaladas; algunas herramientas pueden actualizarse posteriormente.

```
git --version
git config --global user.name
git config --global user.email
node -v
npm -v
py --version
codex --version
flutter --version
wsl --status
wsl --list --verbose
docker --version
docker run hello-world
```

Componente	Comprobación satisfactoria	Si falla
Git	Se muestra la versión y la identidad global es correcta.	Revisar instalación, PATH y `git config --global`.
Node.js / npm	`node -v` y `npm -v` devuelven versiones.	Abrir una terminal nueva; revisar instalación LTS y PATH.
Python	`py --version` muestra Python 3.13.15.	Revisar el lanzador `py` y la instalación de Python.
Codex CLI	Se muestra la versión y puede iniciarse la interfaz.	Reabrir terminal, revisar PATH, autenticación y permisos.
Archify	El gestor detectó e instaló las skills elegidas.	Volver a ejecutar el comando y revisar selección, red y avisos.
Flutter	`flutter --version` funciona; `flutter doctor` informa del estado.	Revisar extracción del SDK y PATH.
WSL / Ubuntu	Ubuntu aparece en `wsl --list --verbose`.	Ejecutar `wsl --install`, reiniciar si se solicita y revisar características de Windows.
Docker	`docker run hello-world` finaliza correctamente.	Iniciar Docker Desktop y revisar el estado de WSL 2.

12. Solución de problemas frecuentes

Síntoma	Acción recomendada
El comando no se reconoce	Cerrar y abrir CMD/PowerShell; verificar PATH y que la instalación haya finalizado.
Winget solicita términos de una fuente	Leer el aviso de términos y confirmar solo si se está de acuerdo; volver a intentar si la descarga falla.
Codex no aparece tras instalar	Abrir una nueva sesión de PowerShell; la evidencia indica que PATH se actualiza para sesiones futuras.
`python` **no funciona pero** `py` **sí**	Usar `py --version` y revisar la asociación de comandos y PATH de Python.
Docker indica que WSL no está instalado	Ejecutar `wsl --install` en PowerShell como administrador, completar Ubuntu y reiniciar si se solicita.
Ubuntu rechaza la respuesta de telemetría	Responder con una opción válida (`y` o `n`) y pulsar Enter; no repetir caracteres.
Flutter Doctor muestra advertencias	Instalar solo las dependencias necesarias para la plataforma objetivo y consultar el detalle de `flutter doctor`.

13. Consideraciones de seguridad y mantenimiento

- Mantenga Windows, Docker Desktop, WSL y los SDK actualizados mediante sus canales oficiales.
- Revise el origen de los scripts antes de ejecutarlos y evite ejecutar comandos copiados de fuentes desconocidas.
- No almacene contraseñas, tokens o claves de acceso en repositorios Git.
- Instale únicamente las skills y agentes que necesita; lea las alertas del gestor antes de confirmar.
- Use Docker con imágenes confiables y evite ejecutar contenedores privilegiados sin una necesidad justificada.
- La instalación correcta de una herramienta no implica que todas sus dependencias opcionales estén configuradas; revise las salidas de diagnóstico correspondientes.

Referencias oficiales

- Git for Windows: https://git-scm.com/download/win
- Node.js: https://nodejs.org/
- Python: https://www.python.org/downloads/
- OpenAI Codex: https://chatgpt.com/codex/
- Flutter: https://docs.flutter.dev/install
- Docker Desktop: https://www.docker.com/products/docker-desktop/
- Microsoft WSL: https://learn.microsoft.com/windows/wsl/install

Fin del documento